Advanced Topics: Revisiting Sequence Models, Transformers, and Large Language Models

Applied Deep Learning — Chapter 25

Yin Yang

Foundation Books Project

Roadmap

Why Revisit Architectures?

The Running Example

A small study-note assistant: a learner asks a question; the system reads long study notes and answers.

- Search or read long notes.
- Keep the learner's question in mind.
- Generate a correct answer with citations.
- Respond fast enough that learners do not abandon the page.

Not a frontier benchmark — exactly the kind of applied constraint that decides whether an architecture is useful.

Applied Architecture Choice

Definition
A decision between model or serving designs for a fixed task, data source, evaluation rule, and deployment setting.

A useful choice reports:

- baseline and measured quality
- training or adaptation time
- inference latency, memory, and cost
- known risks and accepted tradeoffs

It does not collapse to a single universal score — it forces the project to name the tradeoff.

Quality vs Latency: A Concrete Case

Two assistants:

- **A:** 83/100 correct, median latency 0.8 s.
- **B:** 87/100 correct, median latency 4.9 s.

Which is better?

- **Study help page:** four extra correct answers may be worth the delay — pick B.
- **Live chat widget:** A is fast enough to serve more learners under the same server budget — pick A.

Architecture choice is a budget decision.

Old Principles, New Bottlenecks

Modern architectures revisit older ideas because each solves a different bottleneck:

Recurrence Compact state updated as a sequence streams by.

Attention Direct comparisons between positions; can cost memory.

Residuals Stable signal flow through many layers.

Mixture of experts Add capacity by routing tokens to a subset of expert networks.

Measurement Vocabulary

Serving Token Budget

Separate prompt length from generated length:

$$T_{\text{total}} = T_{\text{prompt}} + T_{\text{gen}}.$$

Study-assistant example: 800 prompt tokens (system + user + retrieved passages) plus 200 generated tokens = a 1,000-token interaction.

Different costs grow in different phases — prefill work, cache memory, and output cost are not the same line in the bill.

Latency vs Throughput

Definitions
Latency = end-to-end wall-clock time from request to usable answer (TTFT for streaming).
Throughput = answers or generated tokens per second under stated load and batching.

- A server that batches 64 requests can have high total throughput.
- Yet a single learner request may still wait too long.
- **Median latency** = typical experience.
- **p95 latency** = the slow tail users notice.

The Five-Quantity Report

Quantity	What it asks	Example field
Quality Q	Did the answer solve the task?	Exact match, rubric, citation recall
Training time	How long did fitting take?	42 min on one GPU
Latency	How long did one user wait?	median 0.8 s, p95 2.4 s
Memory	How much state stored?	1.1 GB KV cache at $T = 8{,}192$
Cost	Cost per usable output?	\$ per 1,000 successful answers

A model comparison without these is incomplete.

RNN Revisited

Compact Recurrent State

Definition
A fixed-width summary updated once per position:

$$h_t = F_\theta(h_{t-1}, x_t), \qquad \hat{y}_t = G_\theta(h_t).$$

- State size does not grow with prefix length.
- Work grows roughly linearly with T: 1,000 tokens $\rightarrow \approx$ 1,000 updates.
- The state is also a bottleneck: information not encoded in h_t cannot be recovered later.

Why Modern Recurrence?

Old failure: gradients shrink or explode through repeated state updates. LSTMs/GRUs added gates — learned controls for write, keep, forget.

Modern return is *not* to weak simple RNNs. The applied question:

> *Can a model process very long sequences with compact memory while still learning content-sensitive behavior?*

Two answers:

- State-space models: S4, Mamba.
- Scaled gated memory: xLSTM.

If short Transformers are fast enough, recurrence may be unnecessary.

Mamba and Selective State Spaces

Discrete State-Space Layer

Simplified form

$$s_t = A_t s_{t-1} + B_t x_t, \qquad y_t = C_t s_t.$$

- If A_t, B_t, C_t are fixed: every token treated the same.
- **Selective** state-space (Mamba): controls depend on the input token.

In a study note: "deadline," "optional," and "however" should be treated differently. The model preserves rules; filler can fade.

Selective Update Sketch

Controls become token-dependent:

$$\Delta_t = \text{softplus}(W_\Delta x_t), \quad B_t = W_B x_t, \quad C_t = W_C x_t.$$

Discretization view:

$$\bar{A}_t = \exp(\Delta_t A), \qquad s_t = \bar{A}_t s_{t-1} + \bar{B}_t x_t.$$

Selective scan = efficient algorithmic organization of this recurrence; no full $T \times T$ attention matrix.

Attention vs Selective Scan: The Counts

Full attention

pairwise scores grow as T^2

$T = 1{,}000 \Rightarrow 10^6$
$T = 10{,}000 \Rightarrow 10^8$

Selective scan

one update per token
grows as T

$T = 1{,}000 \Rightarrow 10^3$
$T = 10{,}000 \Rightarrow 10^4$

Counts $\neq$ wall-clock time. Hardware kernels matter; Mamba's paper emphasizes a hardware-aware parallel algorithm.

When to Try Mamba?

- **Worth trying:** long context or streaming input is the bottleneck.
- **Less obvious:** when the main problem is exact retrieval — a retrieval system may be simpler and easier to audit.

Honest experiment: small Transformer baseline, retrieval-augmented baseline, and state-space model on the same questions; report accuracy, latency, and memory together.

If the answer is wrong because the relevant note was never retrieved, a faster sequence layer is solving the wrong problem.

xLSTM and Scaled Gated Memory

Modern Gated-Memory Update

$$c_t = f_t \odot c_{t-1} + i_t \odot u_t.$$

- c_t: memory state. u_t: candidate new info.
- f_t: forget gate. i_t: input gate. $\odot$: elementwise product.

Examples:

- $f_t = 0.90$, $i_t = 0.20$: mostly preserve, small update.
- $f_t = 0.10$, $i_t = 0.85$: mostly replace.

Gates are controls, not just notation.

xLSTM Innovations

Exponential gating before normalization:

$$\tilde{i}_t = \exp(a_t), \qquad \tilde{f}_t = \exp(b_t).$$

Matrix-memory variant (outer-product update):

$$M_t = f_t \odot M_{t-1} + i_t \odot (v_t k_t^\top), \qquad o_t = M_t q_t.$$

k_t, v_t, q_t play key/value/query roles. Not just the old LSTM with a new name — redesign with modern scaling, normalization, and residual blocks.

Mamba vs xLSTM: Same Question, Different Tools

Mamba

- Selective state-space dynamics.
- Hardware-aware scans.
- Linear-time over T.

xLSTM

- Scaled recurrent memory.
- Modern gates and normalization.
- Residual block design.

Both ask the same applied question: under same data and budget, which model gives acceptable quality at lowest latency and memory cost?

Transformer Revisited: Attention and KV Cache

Full Attention Score Matrix

Definition
For T token reps with $Q, K \in \mathbb{R}^{T \times d}$:

$$S = QK^\top \in \mathbb{R}^{T \times T}.$$

Pairwise scores per head scale as T^2.

Counts:

- $T = 100 \Rightarrow 10{,}000$ scores.
- $T = 1{,}000 \Rightarrow 1{,}000{,}000$.
- $T = 10{,}000 \Rightarrow 100{,}000{,}000$.

Long context is not just a larger input box.

KV Cache During Decode

Decoder-only Transformers store keys and values; do not recompute.

Memory estimate

$$M_{\mathrm{KV}} \approx 2 \cdot L \cdot T \cdot H_{\mathrm{kv}} \cdot d_h \cdot b.$$

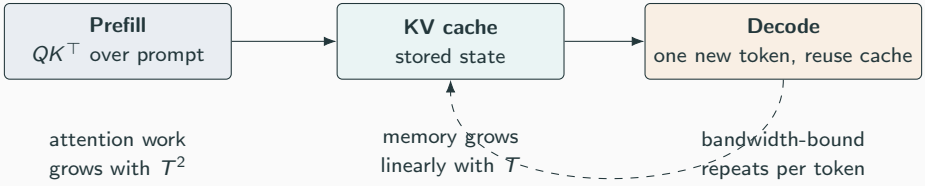
Factor 2: keys and values both stored.

Toy: $L = 12,\ T = 4{,}096,\ H_{\mathrm{kv}} = 8,\ d_h = 64,\ b = 2$:

$$2 \cdot 12 \cdot 4096 \cdot 8 \cdot 64 \cdot 2 \approx 96 \text{ MiB}.$$

Doubling context to $T = 8{,}192$ doubles cache to ≈ 192 MiB per sequence.

Two Phases, Two Bottlenecks



Efficient-Attention Variants

Different ideas attack different bottlenecks:

MQA Multi-query attention: shared K, V across query heads; reduces decode bandwidth.

GQA Grouped-query attention: middle ground between separate and shared K, V.

MLA Multi-head latent attention (DeepSeek-V3): compress K, V into a latent.

FlashAttention Exact attention; reduces GPU memory traffic via IO-aware algorithm.

Sparse / sliding Reduce which positions are compared.

RoPE / context extension Position handling at long T.

Choose because the variant attacks a measured bottleneck.

Residual Streams and Hyper-Connections

Why Residual Streams Matter

Standard residual form:

$$z_{\ell+1} = z_\ell + F_\ell(z_\ell).$$

- Identity-like path through depth.
- If F_ℓ is small at init, info still flows forward.
- In the assistant: a representation of the learner's question can survive many layers while being refined.

Without stable paths, training becomes fragile: activations or gradients shrink, explode, or become poorly conditioned.

Hyper-Connections (HC) and mHC

HC carries K residual streams that can mix:

$$S_{\ell+1} = M_\ell S_\ell + F_\ell(S_\ell), \qquad S_\ell \in \mathbb{R}^{K\times d}.$$

Concern: arbitrary M_ℓ can amplify or collapse directions through depth.

Manifold-constrained HC (DeepSeek mHC) restricts M_ℓ:

$$M_\ell^\top M_\ell \approx I.$$

Plain language: richer residual routes should not destroy stable signal flow.

Residual-Stream Stability

Definition
Reps and gradients pass through many layers without systematic explosion or collapse.

Practical signs of *instability*:

- Repeated loss spikes.
- Numerical overflow.
- Failure to make progress where a comparable residual model trains normally.

HC/mHC is not the first knob to turn when answers are wrong. Check data, retrieval, prompt, and ordinary model choice first.

Mixture of Experts

Dense vs Sparse Active Parameters

Definition
Dense: same parameter set for each token.
Sparse MoE: total P_{total}, but only $P_{\text{active}}(x_t) \ll P_{\text{total}}$ active per token.

Toy comparison:

- Dense: 80M params, all used per token.
- MoE: 240M total, but router activates 60M per token.

3× total capacity, fewer active params per token. This is the basic capacity-computation tradeoff.

MoE Routing

Router scores $r_\theta(x_t)$. Top-k layer:

$$S_t = \mathrm{TopK}(r_\theta(x_t), k), \qquad y_t = \sum_{e \in S_t} \alpha_{t,e}\, E_e(x_t).$$

4-expert layer, top-2, 6-token prompt $\Rightarrow 6 \times 2 = 12$ expert calls.

- All to experts 1&2: experts 3&4 idle, system poorly balanced.
- Spread evenly: hardware used effectively.

Load balancing is part of the architecture, not an afterthought.

MoE: Promise and Pitfalls

Attractive

- More total capacity.
- No dense compute per token.
- Specialization across experts.

Awkward

- Experts on different devices.
- Routing creates communication.
- Some experts overloaded; others idle.
- Low FLOPs may still mean high latency.

Report total params, active params, and latency together.

Inference Systems

Prefill and Decode

Definition
Prefill: prompt processing; builds activations and initial KV-cache entries before first generated token.

Decode: repeated generation; each step uses cache, emits new tokens, appends new cache entries.

Study assistant: prompt of 6,000 tokens, answer of 200 tokens.

- **Prefill bottleneck:** attention work over the prompt.
- **Decode bottleneck:** cache memory and bandwidth.

Optimizing prefill does not automatically speed decode.

Quantization: A Two-Sided Tool

Store/compute values at lower precision (8-bit, 4-bit) to save memory and increase speed.

Risk: lower precision can hurt quality or numerical stability.

Incomplete report: "The 4-bit model was faster."

Applied report: "The 4-bit model reduced median latency from 2.6 s to 1.5 s while lowering validation answer accuracy from 86% to 84%."

Always pair latency change with quality change.

Speculative Decoding

- Small draft model proposes several tokens.
- Larger target model verifies in a way that preserves the target distribution.

Toy: 24-token answer, ordinary decoding $=$ 24 large-model calls.

If verification accepts an average of 3 draft tokens:

$$24/3 = 8 \text{ large-model verification calls.}$$

Real speedup is smaller (draft $+$ verify have costs), but the calculation explains the appeal.

The Practical Serving Report

Include all of:

- median latency, p95 latency
- throughput, batch size
- context length, output length
- precision (fp16, int8, int4, . . .)
- hardware or API backend
- quality on a fixed validation set

Without these the claim cannot be reproduced. With them, even a small classroom experiment can teach the central applied lesson.

Architecture Decision Audit

A Worked Decision Audit

Task: 100 held-out study-note questions, fixed retrieval and rubric.

Run	Quality	Median	p95	What it proves
Baseline Transformer	83/100	0.8 s	2.4 s	Fast, misses long-context evidence
Long-context attn.	87/100	4.9 s	11.8 s	Quality up; interactive latency suspect
Speculative decoding	83/100	0.55 s	1.5 s	Serving win at fixed quality

Not a leaderboard. A decision record.

Reading the Audit

- **Offline grading assistant:** long-context variant may win — 4 extra correct answers $>$ a few seconds.
- **Live help widget:** speculative decoding wins — faster wait at the same answer score.
- **MoE / Mamba / xLSTM:** enter the table only after naming the bottleneck they are meant to fix.

Unsupported claims to flag: "long-context is generally better," "speculative decoding is always safe," "recurrent state would fail."

The audit proves a narrow, useful point.

Summary

Recurring Questions, Different Answers

Architectures answer four recurring questions:

1. How should information be **remembered**?
2. How should positions **interact**?
3. How should signals pass through **depth**?
4. How much of the model should be used for **one token**?

- Mamba / xLSTM: compact memory for long sequences.
- Efficient attention (MQA/GQA/MLA, FlashAttention): direct comparison without quadratic blowup.
- HC / mHC: stable signal flow at depth.
- MoE: conditional computation, total $\neq$ active.

The Mini Audit Habit

Before trying an advanced block, write one line each for:

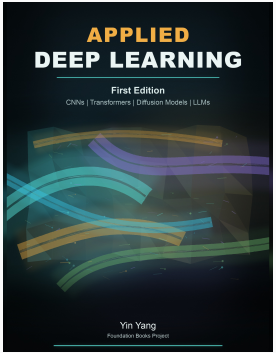
- baseline failure
- bottleneck category
- proposed method
- fixed comparison controls
- keep-or-drop evidence
- claim the experiment still cannot support

Then ask: what is slow, unstable, too expensive, or too inaccurate? Choose the smallest experiment that could change that evidence.

Choose the next architecture experiment from the measured bottleneck.

Compare against a baseline.

Report quality together with speed, memory, cost, and limitations.



Applied Deep Learning

Chapter overview slides for the book by Yin Yang.

Book page	`foundation-books.github.io/applied-deep-learning`
Code	`github.com/foundation-books/applied-deep-learning-code`
Print	`amazon.com/dp/B0GZF7QRSH`
Kindle	`amazon.com/dp/B0GZF9221X`
License	CC BY-NC-ND 4.0

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.